

Smart City Air Quality Monitoring System

End-to-end data engineering pipeline for five Pakistani cities

Project Documentation

Program: Data Engineering Hackathon — Batch 05

Instructor: Ayan Hussain

Prepared by: Taha Khan

Cities covered: Karachi, Lahore, Islamabad, Peshawar, Multan

Stack: Python · Snowflake · Streamlit · OpenAQ v3 API

1. Overview

The Smart City Air Quality Monitoring System is an end-to-end data pipeline that measures and visualizes air quality across five major Pakistani cities. It combines two data sources — a Python-based IoT sensor simulator and real reference data from the public OpenAQ v3 API — and processes them through a Snowflake medallion architecture (Bronze, Silver, Gold) before serving the results on an interactive Streamlit monitoring dashboard.

The system computes the US EPA Air Quality Index (AQI) from PM2.5 concentrations, classifies every reading by category and health-risk level, detects pollution anomalies, and presents live metrics, trends, a station map, and plain-language health advisories.

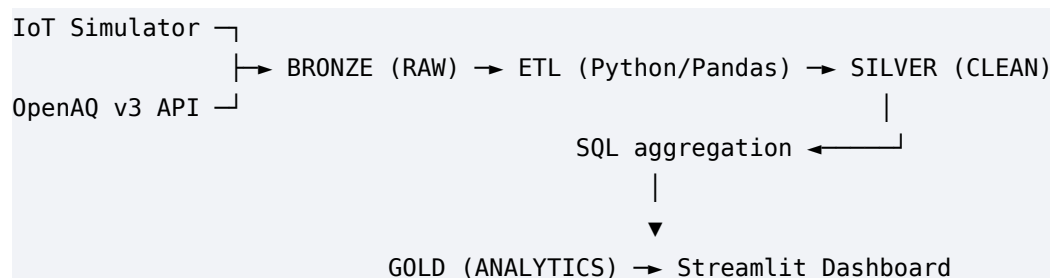
2. Problem & Objective

Air pollution is a serious and worsening public-health issue in Pakistani cities, yet real-time, consolidated, and easy-to-understand air-quality information is limited. Raw sensor and station data is fragmented and technical, making it hard for citizens and authorities to act on.

The objective of this project is to demonstrate a production-style data engineering workflow that ingests high-frequency sensor data alongside real-world reference measurements, cleans and standardizes it, derives meaningful health indicators, and delivers clear, actionable insight through a live dashboard.

3. System Architecture

The pipeline follows the medallion pattern. Both sources land independently in the Bronze (raw) layer. The Silver layer validates, enriches, and unifies them into a single clean table. The Gold layer aggregates the data into daily per-city summaries that power the dashboard.



4. Data Sources

4.1 IoT Sensor Simulator

A Python simulator generates realistic readings for ten virtual sensors (two per city) every ten seconds. Pollution baselines depend on zone type; readings include a time-of-day effect (traffic peaks near 8 AM and 6 PM), $\pm 15\%$ random noise, and a 15% chance of an anomaly spike (PM2.5 multiplied by 2.5–4.0) representing real pollution events.

Sensor network:

Sensor ID	City	Zone type
PKS_KHI_IND_01	Karachi	Industrial
PKS_KHI_TRF_02	Karachi	Traffic
PKS_LHR_RES_01	Lahore	Residential
PKS_LHR_IND_02	Lahore	Industrial
PKS_ISB_PRK_01	Islamabad	Park
PKS_ISB_TRF_02	Islamabad	Traffic
PKS_PEW_IND_01	Peshawar	Industrial
PKS_PEW_RES_02	Peshawar	Residential
PKS_MUL_TRF_01	Multan	Traffic
PKS_MUL_PRK_02	Multan	Park

Zone baseline ranges (PM2.5 $\mu\text{g}/\text{m}^3$):

Zone	PM2.5	CO ₂ (ppm)	Temp (°C)
Industrial	80–120	600–900	30–42
Traffic	55–80	500–700	28–40
Residential	25–50	420–500	25–38
Park	8–20	400–430	22–35

4.2 OpenAQ v3 API

Real reference data is pulled from the public OpenAQ v3 API for Pakistan. The fetcher retrieves all Pakistan stations, maps each station's sensors to their parameters, and collects the latest PM2.5 and PM10 measurements. Requests are authenticated with an API key and rate-limited to respect the free tier (60 requests/minute).

5. Data Pipeline (Medallion Layers)

Layer	Object	Purpose
Bronze	RAW.IOT_READINGS	Raw simulated sensor readings, stored as-is
Bronze	RAW.OPENAQ_RAW	Raw OpenAQ pollutant readings, stored as-is
Silver	CLEAN.AQI_CLEAN	Validated, enriched, unified data from both sources
Gold	ANALYTICS.CITY_DAILY	Daily per-city aggregates for the dashboard

5.1 Transformations (Bronze → Silver)

- **IoT:** drop null PM2.5/AQI; validate ranges (PM2.5 0–500, CO₂ 400–2000, humidity 0–100); derive AQI category and health risk; de-duplicate on (sensor_id, recorded_at); attach city coordinates.
- **OpenAQ:** keep PM2.5/PM10 only; convert timestamps to UTC; drop non-positive values; pivot to a wide format; derive AQI from PM2.5.
- **Unification:** both sources are concatenated into one Silver table with a common schema and a source tag.

5.2 AQI Calculation

The US EPA linear-interpolation formula converts a PM2.5 concentration (C) into an AQI value:

$$\text{AQI} = (I_{hi} - I_{lo}) / (C_{hi} - C_{lo}) * (C - C_{lo}) + I_{lo}$$

Each reading is then classified into a category (Good, Moderate, Unhealthy for Sensitive, Unhealthy, Very Unhealthy, Hazardous) and a health-risk level (LOW, MEDIUM, HIGH, CRITICAL). The same logic is shared by the simulator and the ETL so values remain consistent.

5.3 Aggregation (Silver → Gold)

A SQL statement aggregates the Silver table into daily per-city metrics: average, maximum and minimum AQI; average PM2.5 and CO₂; the dominant health-risk level (statistical mode); and the reading count.

6. Dashboard

The Streamlit dashboard reads the Gold and Silver layers from Snowflake (with an automatic fallback to local files so it always runs during a demo). It adopts a dark monitoring-console design using the EPA AQI colour spectrum. Key features:

- **Live snapshot KPIs** — highest-AQI city, total readings, anomalies in the last hour, and average PM2.5.
- **AQI gauge** — an instrument-style gauge showing the worst city's air quality against EPA colour bands.
- **Health advisory** — a plain-language recommendation based on the current worst reading.
- **Average AQI per city** — a colour-coded bar chart.
- **Station map** — Pakistan-centred map with stations coloured by AQI.
- **6-hour trend** — per-sensor AQI lines with detected anomalies marked.
- **Latest readings** — a colour-coded severity table.
- **Live auto-refresh** — the dashboard updates every 30 seconds for a real-time monitoring experience.

7. Technology Stack

Area	Tools
Language	Python 3
Data processing	Pandas, NumPy
Data warehouse	Snowflake (Bronze / Silver / Gold)
External data	OpenAQ v3 REST API
Dashboard	Streamlit, Plotly
Config / secrets	python-dotenv (.env)
Version control	Git / GitHub

8. How to Run

```
pip install -r requirements.txt
cp .env.example .env          # add OpenAQ key + Snowflake creds
# create schema: run sql/snowflake_schema.sql (DDL) in Snowflake
python iot_simulator.py --minutes 30 --to-snowflake
python openaq_fetcher.py --to-snowflake
python etl_pipeline.py --from-snowflake --to-snowflake
# run the Silver -> Gold INSERT in snowflake_schema.sql
streamlit run dashboard/app.py
```

9. Deliverables

- IoT simulator, OpenAQ fetcher, ETL pipeline, Snowflake SQL, and dashboard code
- README with setup and run instructions
- Screenshots of all deliverables
- 5-minute live demo: run the simulator, show data flowing into Snowflake, open the dashboard

10. Future Enhancements

- Automated scheduling (e.g. Airflow) for continuous ingestion and aggregation
- Historical trend analysis and short-term AQI forecasting
- Alerting (email/SMS) when a city crosses the Hazardous threshold
- Zone-level and pollutant-level breakdown views